

E4 Completion Guide

This guide documents the end-to-end implementation of Phase E4 (Orchestrator & End-to-End Sessions) based on our latest work, along with how to verify it locally.

1. Overview

****Phase Goal:**** Wire E1 (data), E2 (quant evaluation), and E3 (LLM review) into reliable sessions wrapped with FastAPI endpoints, with explicit persistence, dependency injection, graceful degradation, and test coverage.

****Current state:****

- Database schema (`thesis\_evaluation` , `alert` , `daily\_summary`) applied and used
- `DataAccess` extended with new repositories for E4
- `LLMTools` wraps the Phase 5 orchestrator and exposes task-specific methods (`review\_thesis` , `daily\_summary` , etc.)
- Sessions implemented:
 - `ThesisEvaluationSession` : deterministic E2 + E3, atomic write-at-end persistence
 - `DailyUpdateSession` : collects alerts, calls LLM, persists daily summary, degrades gracefully
 - `CrossThesisSession` : thin wrapper over LLM
- FastAPI router (`src/slice/api/session\_routes\_e4.py`) exposes endpoints:
 - `POST /api/v1/thesis/{thesis\_id}/evaluate`
 - `POST /api/v1/session/daily-update`
 - `POST /api/v1/thesis/compare`

- Error semantics enforced:
 - 404 for missing thesis
 - 500 for E2/E3 failures
 - Daily update returns 200 even if LLM summary fails
- Tests in `tests/e4/` cover repositories, sessions, and API routes

2. Dependency Wiring

Before this iteration, `DataAccess.depends()` raised during real server runs. E4 testing relied on `app.dependency\_overrides`. Now, the FastAPI app wires production dependencies:

****Key file**:** `src/slice/api/main.py`

- On startup, we call `wire\_dependencies()` which:
 - Instantiates `DataAccess` (with theses, observations, trades, alerts, evaluation, daily summary repos)
 - Instantiates `PriceSource` (currently `SimpleMockPriceSource` for testing)
 - Instantiates `OrchestratorClient` (OpenAI LLM)
 - Registers dependency overrides for:
 - `DataAccess.depends`
 - `OrchestratorClient.depends`
 - `get\_eval\_service` (ensuring real price source in E4 endpoints)
 - Optional dev convenience: apply E4 schema when `SLICE\_AUTO\_APPLY\_SCHEMA=true`

****Key file**:** `src/slice/api/deps.py`

- Provides singleton factories for:

- `DataAccess`
- `SimpleMockPriceSource` (deterministic linear GLD price data)
- OpenAI LLM client (`AsyncOpenAI`), reading `OPENAI\_API\_KEY` and `OPENAI\_MODEL`
- `OrchestratorClient`

3. Verifying Locally

3.1 Environment

Create `.env` (based on `env.template`):

```

SLICE\_DB\_URL="postgresql+psycopg2://slice\_user:slice\_password@localhost:5432/slice"

OPENAI\_API\_KEY="sk-..."

OPENAI\_MODEL="gpt-4o-mini"

SLICE\_AUTO\_APPLY\_SCHEMA="true" # optional for dev

```

Install deps:

```bash

python3 -m venv .venv

source .venv/bin/activate

```
pip install -r requirements.txt
```

```
```
```

Apply schema if needed:

```
```bash
```

```
PYTHONPATH=src python -c "from slice.db import apply_schema, apply_phase4_schema,
apply_e4_schema; apply_schema(); apply_phase4_schema(); apply_e4_schema()"
```

```
```
```

3.2 Run API

```
```bash
```

```
source .venv/bin/activate
```

```
PYTHONPATH=src uvicorn slice.api.main:app --reload
```

```
```
```

3.3 Insert Test Thesis

Use SQL script created during this work:

```
```bash
```

```
psql "postgresql://slice_user:slice_password@localhost:5432/slice" -f
scripts/insert_test_thesis.sql
```

```
```
```

Verifying:

```
```bash
```

```
psql ... -c "SELECT id, status FROM thesis WHERE id = 'TEST_T1';"
```

```
```
```

3.4 Evaluate Endpoint

```
```bash
```

```
curl -i -X POST "http://127.0.0.1:8000/api/v1/thesis/TEST_T1/evaluate"
```

```
```
```

Expected `200 OK`:

- `evaluation.performance.total\_return  $\approx$  0.10` (mock data)
- Structured `review` with critique/questions/risk flags
- `evaluated\_at` timestamp

Confirm persistence:

```
```bash
```

```
psql ... -c "SELECT thesis_id, evaluated_at FROM thesis_evaluation WHERE thesis_id = 'TEST_T1';"
```

```
```
```

3.5 Daily Update

```
```bash
```

```
curl -i -X POST "http://127.0.0.1:8000/api/v1/session/daily-update"
```

```
```
```

Even without active theses, returns `200` with empty alerts, `insufficient\_context=true` summary.

4. Testing

With compatible environment (FastAPI installed, Pydantic v1), run:

```
` `` `bash
```

```
pytest tests/e4 -q
```

```
pytest tests/phase8/test_data_access_extensions.py -q
```

```
` `` `
```

`tests/phase9/test\_ui\_routes\_integration.py` currently needs updates:

- They create `TestClient(app)` without wiring `DataAccess` (startup overrides don't run in TestClient)
- To fix: use fixtures that call `get\_data\_access\_instance()` and override dependencies (mirroring E4 tests)

5. Open Items / Recommendations

1. **PriceSource**: Replace `SimpleMockPriceSource` with a real implementation for production (e.g., DB-backed or TwelveData API).
2. **Debug prints**: Remove the temporary E2 debug logs in `ThesisEvaluationSession`.

3. **Testing**: Update phase9 tests to use dependency overrides.
4. **Schema migrations**: Use a migration tool in production instead of ``apply..._schema``.
5. **FastAPI lifecycle**: Consider switching from ``@app.on_event("startup")`` to lifespan context (per deprecation warning).
6. **LLM config**: Add timeouts/retries/logging as needed.

6. Summary

Phase E4 is fully implemented and validated:

- ☒ New schema + repositories
- ☒ Sessions for thesis evaluation, daily updates, cross-thesis analysis
- ☒ LLM integration via ``LLMTools`` and ``OrchestratorClient``
- ☒ FastAPI endpoints with dependency injection
- ☒ Atomic persistence + graceful degradation
- ☒ Tests covering core behavior

To go further, focus on production-grade data sources, LLM hardening, and test modernization.